



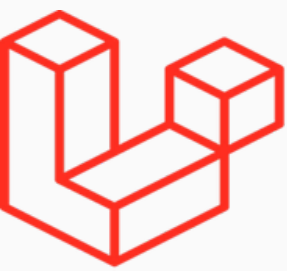
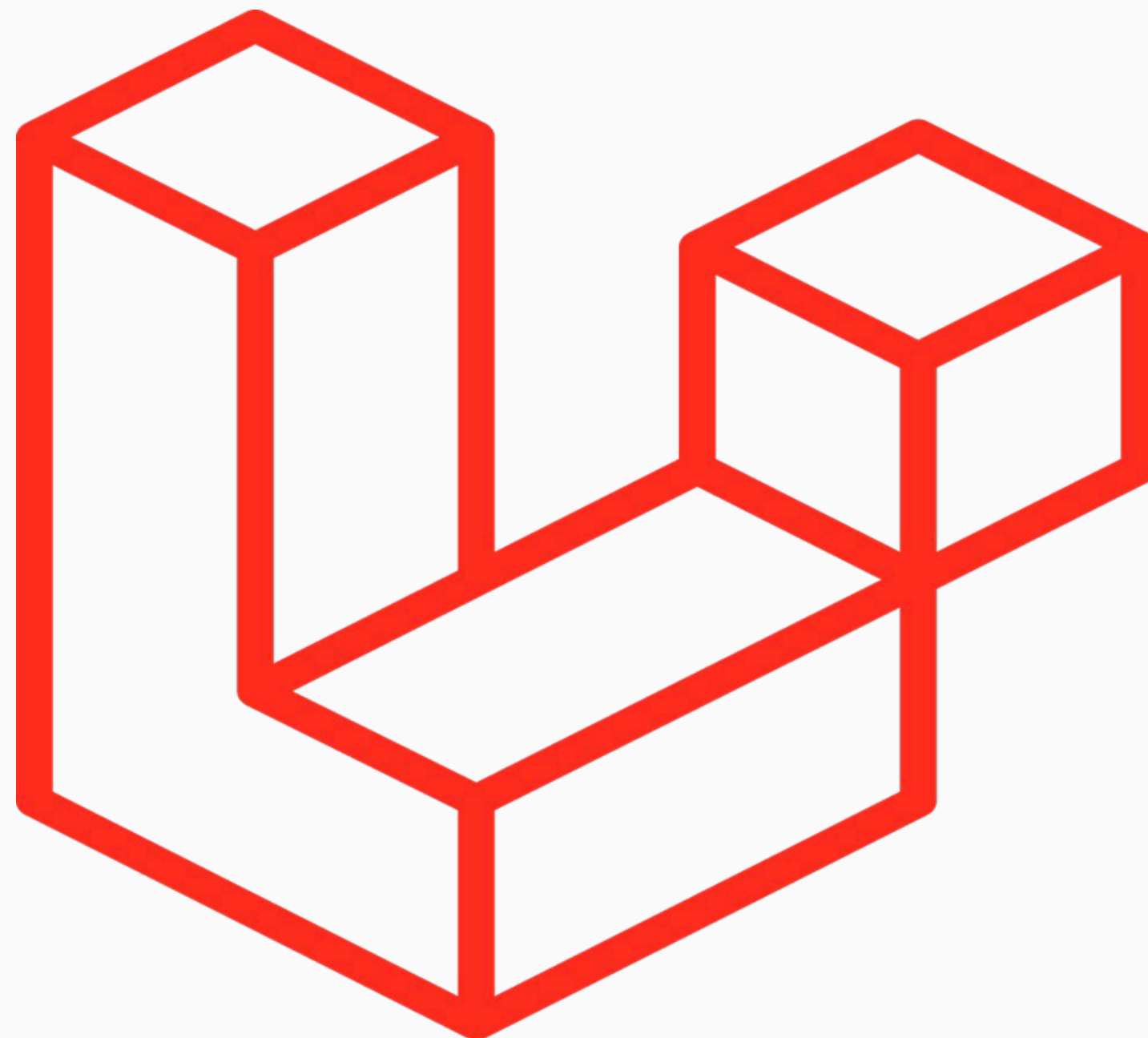
Laravel Course

Day 4



Mona Ali Mostafa

TEACHING ASSISTANT AT ITI

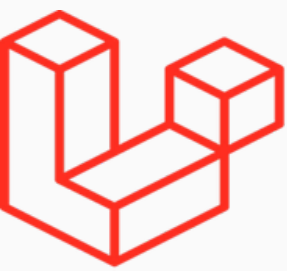


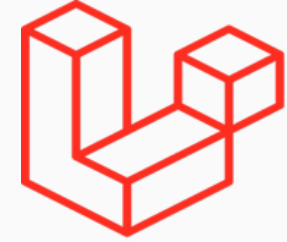


Agenda



- 1 Upload Images
- 2 Authantication
- 3 Laravel Debugger
- 4 Laravel Auth → UI / Breeze
- 5





Uploading & Storing Images

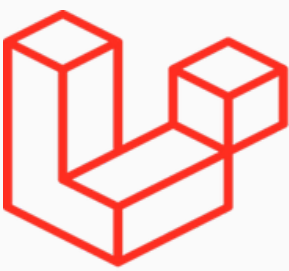
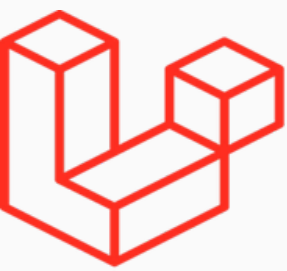
1. Prepare Database

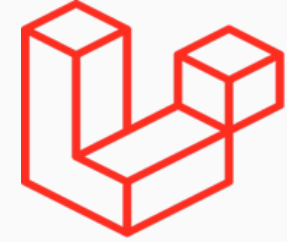
Add an image column to your posts table:

```
php artisan make:migration add_image_to_posts_table
```

```
public function up()
{
    Schema::table('posts', function (Blueprint $table) {
        $table->string('image')->nullable();
    });
}
```

```
php artisan migrate
```





Uploading & Storing Images

2- In Post.php, allow mass assignment:

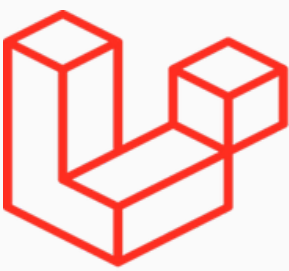
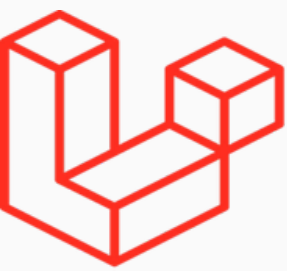
```
protected $fillable = ['title', 'description', 'image'];
```

3. Add file input inside your form:

```
<input type="file" name="image">
```

Important:

- **enctype="multipart/form-data"** is required for file upload





Uploading & Storing Images

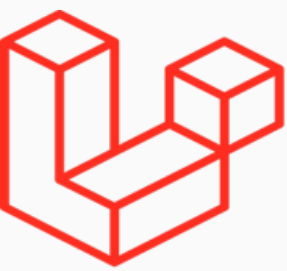
4. Handle Upload in Controller

```
'image' => 'required|image|mimes:jpg,png,jpeg|max:2048'
```

```
$imagePath = null;
```

```
if ($request->hasFile('image')) {  
    $imagePath = $request->file('image')->store('posts', 'public');  
}
```

```
Post::create([  
    'image' => $imagePath,  
]);
```





Uploading & Storing Images

5. Create Storage Link

- Run this command:

```
php artisan storage:link
```

- ✓ This links:
- storage/app/public → public/storage
- storage: where laravel store images, videos, files but not accessible at browser for security reasonsso make link

6-Display Image in Blade

```

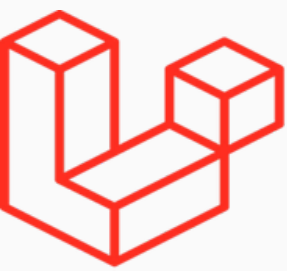
```





Reusable Image Upload Trait

```
namespace App\Traits;
use Illuminate\Http\Request;
use Illuminate\Support\Str;
trait UploadImageTrait {
    public function uploadImage(Request $request, $folder, $fieldname = 'image')
    {
        if ($request->hasFile($fieldname) && $request->file($fieldname)->isValid()) {
            $image = $request->file($fieldname);
            $extension = $image->getClientOriginalExtension();
            $imageName = Str::uuid() . '.' . $extension;
            return $image->storeAs($folder, $imageName, 'public');
        }
        return null;
    }
}
```





Reusable Image Upload Trait

```
use App\Traits\UploadImageTrait;
```

```
class PostController extends Controller  
{
```

```
    use UploadImageTrait;
```

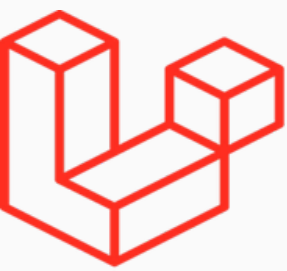
```
    public function store(Request $request)  
    {
```

```
        $data['image'] = $this->uploadImage($request, 'galleries');
```

```
        Post::create($data);
```

```
    }
```

```
}
```





Registration



```
public function register(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|min:6|confirmed'
    ]);
    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);
    auth()->login($user);
    return redirect('/');
}
```





Login

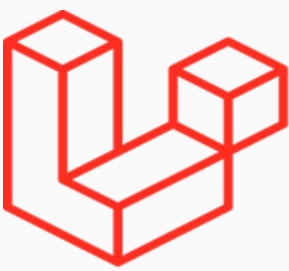
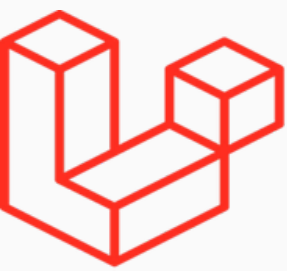


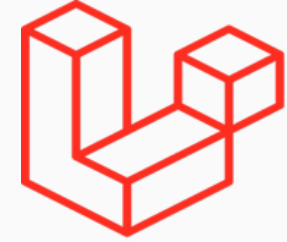
```
public function login(Request $request)
{
    $credentials = $request->only('email', 'password');

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate(); // Regenerating the session ID

        return redirect()->route('/');
    }

    return back()->withErrors([
        'email' => 'Invalid credentials'
    ]);
}
```





Logout



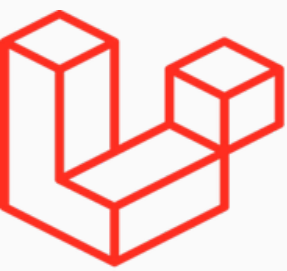
```
public function logout(Request $request)
{
    Auth::logout(); // remove user data from session

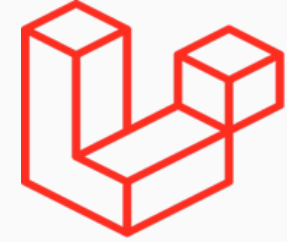
    // regenerate the session ID and remove all data from the session
    $request->session()->invalidate();
    $request->session()->regenerateToken(); // regenerate new csrf token

    return redirect('/login');
}
```

web.php

```
Route::post('/logout', [AuthController::class, 'logout'])
->middleware('auth')
->name('logout');
```





Packages

Laravel Ui

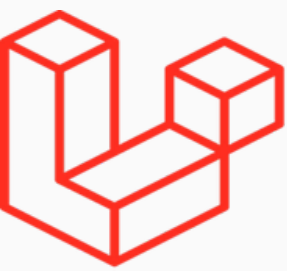
<https://github.com/laravel/ui>

Laravel Breeze

<https://laravel.com/docs/10.x/starter-kits#laravel-breeze-installation>

Laravel Debugger

<https://github.com/fruitcake/laravel-debugbar#installation>





Lab

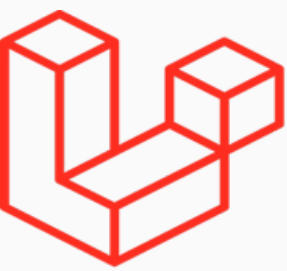


- Use [laravel/Breeze](#) to scaffold the authentication pages.
- Modify the current **navbar** to use Laravel's default authentication navbar, and include a link to All Tasks (adjust the design as needed).
- Add authentication middleware to all task routes so that any unauthenticated user is redirected to the login page.
- Add a **slug** field to the task model using the cviebrock/eloquent-sluggable package.
 - The slug should be automatically generated from the task title.
 - Users are not allowed to send or fill the slug manually in the request (use `$request->validated()` or `$request->only()` appropriately).
 - Read the package documentation carefully:
<https://github.com/cviebrock/eloquent-sluggable>

Example:

Title : Hello Laravel → Slug : hello-laravel

- Display the slug column on the Index page.





Upload an **Multipule Images** to the task, ensuring that only **.jpg and .png** extensions are allowed.

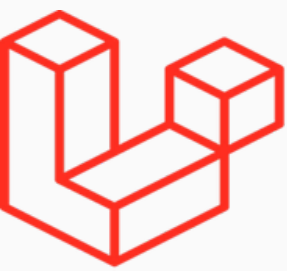
- create table “posts_images” → relations one to many
- Use Laravel validation to restrict file types (jpg, png) when uploading the image.
- Store images using Laravel Storage (e.g., storage/app/public) and display them properly.
- When **updating** a task:
 - Upload the new image, Delete the old image from storage to avoid unused files.
- When **deleting** a task:
 - Also delete the associated image from storage.

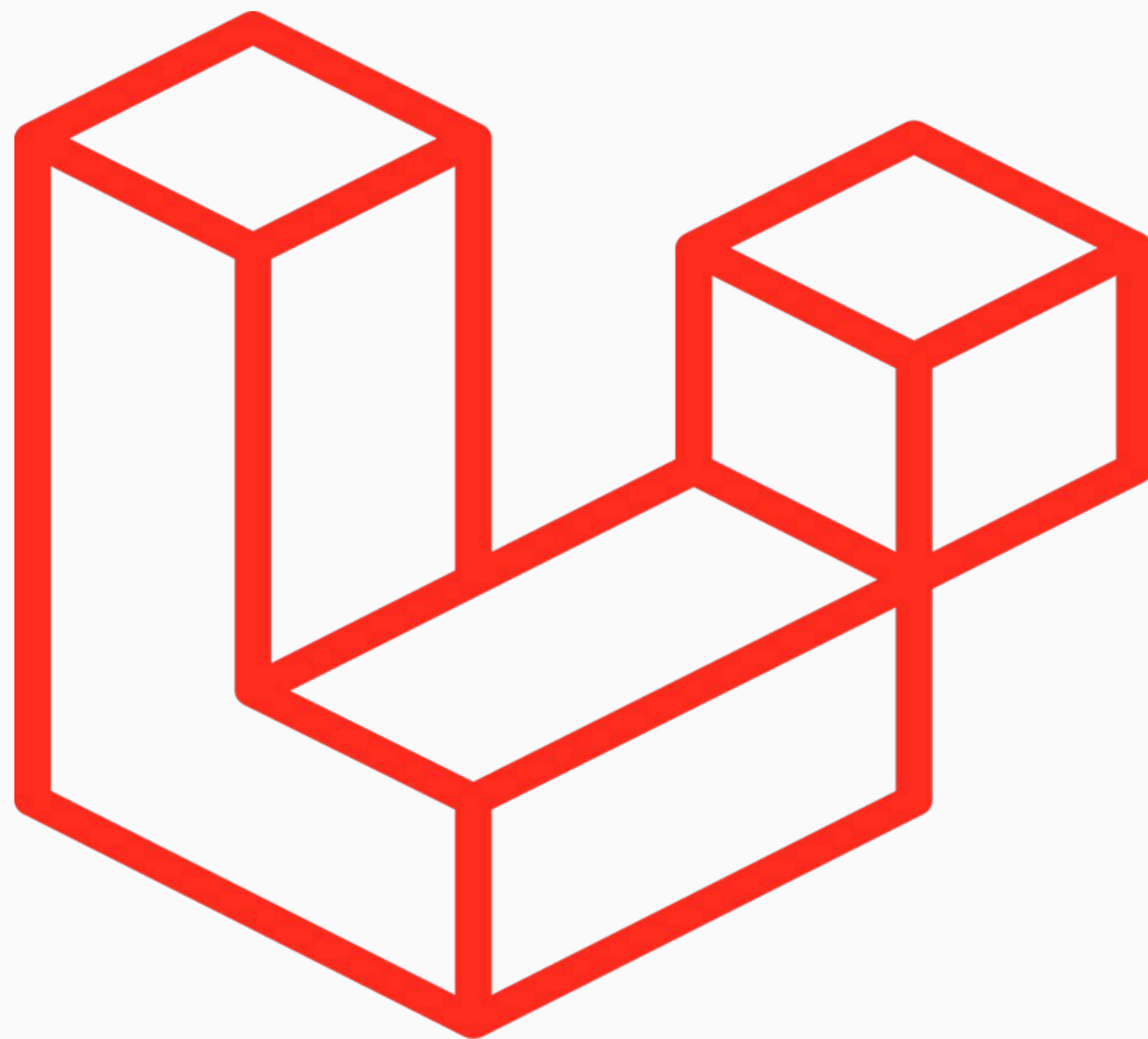
You can refer to the official Laravel documentation:

<https://laravel.com/docs/master/filesystem#file-uploads>

Hint:

- Consider using Mutators (Attribute casting) to handle image paths or default values cleanly and keep your controller logic simpler.
- insted of use this `{{ asset("storage/" . $post->image) }}` → `$post->image_url`





Thank You!

Next: Day 2 – DB Facade & Eloquent -
Database Factory - Database Seeds - Artisan
Tinkere

LARAVEL COURSE – DAY 1